Compete then Collaborate: Frontier AI Teachers Build a Verifiable Curriculum to Improve a Coding Student Beyond Imitation

Miseong (Shawn) Kim
Genesis Cortex AI Inc.
ORCID: 0009-0003-7566-3212

July 2026

Code, data, tests, and verification harness:
`https://github.com/shawnkim678/compete-then-collaborate`
Every number is recomputable from the released files or by regenerating teacher solutions under your own provider terms.

Abstract

Large language models are increasingly used as *teachers* that generate training data for smaller *student* models. Prior multi-teacher knowledge distillation merges several teachers' outputs, but does not ask **which** frontier model teaches best, and typically relies on an LLM judge that is known to favor its own outputs. We introduce a **compete-then-collaborate** framework in which four frontier AI teachers spanning the major labs (Claude/Anthropic, Codex-GPT/OpenAI, Grok/xAI, Gemini/Google) are first **ranked head-to-head** by an *execution-based* judge (unit tests / stdin–stdout checks) with fairness controls (a shared task bank, teacher self-correction, and an intersection-controlled training set), and then **collaborate** to build a *verifiable curriculum* for a single coding student (Qwen2.5-Coder). We report three findings. (1) Under execution verification, all four teachers solve standard problems near-perfectly after self-correction (≈99–100%) — a saturation effect, not a skill difference — while harder competition problems separate them (Gemini 77% > Claude 69% ≈ Codex 69% > Grok 50%); still, the most robust results are on the student side and do not depend on the teacher ranking. (2) **Imitation (SFT) on the teachers' verified solutions does not improve — and can degrade — an already-competent coder student** at both 7B and 32B (e.g., 76.7%→72.7% on MBPP-test, 5.9%→2.9% on competition problems for the union of all teachers). (3) The *same* collaborative curriculum used as a **reinforcement-learning-with-verifiable-rewards (RLVR)** environment instead **improves** the student (5.9%→8.8% peak on held-out competition problems over a 1000-step run, +49% relative), reversing the direction of SFT. Our central claim is that the value of AI-teacher collaboration is **not** pooling answers to imitate, but jointly constructing a verifiable environment in which the student learns by doing. We release a fully reproducible on-prem pipeline (NVIDIA GB10) including framework patches required to run GRPO on a bleeding-edge stack.

1 Introduction

Distilling capable "teacher" LLMs into smaller "student" models is now standard practice. Two questions are under-explored: (i) *which* commercial frontier model is the better teacher, measured by the student's real downstream ability rather than by an LLM judge; and (ii) whether *combining* teachers is best done by merging their answers or by some other mechanism.

We study these on Python coding. Our judge is **code execution** — objective and free of the self-preference bias documented for LLM-as-judge setups. We first run a **competition**: teachers solve a shared task bank; a teacher's output enters the student's data only if it passes hidden tests. We then run a **collaboration**: all teachers' verified work forms one curriculum, used two ways — as imitation targets (SFT) and as a verifiable reward environment (RLVR).

Contributions. (1) An execution-verified, bias-free **ranking of frontier AI teachers** with three fairness controls (shared tasks, self-correction, intersection). (2) A controlled **comparison of collaboration modes** showing imitation-SFT fails/degrades competent coder students while verifiable-reward RL improves them. (3) A **reproducible on-prem (GB10) pipeline**, including the framework patches needed to run GRPO on transformers 5.5 / torch 2.11 / cu130.

2 Related Work

Multi-teacher KD. Prior work merges multiple teachers' rationales into a student and reports that naively adding teachers can *hurt* due to *knowledge conflict*, motivating purification/consolidation [3]. Our union-SFT result independently reproduces this degradation, but our framing differs: we do not merge to imitate; we build a verifiable environment.

Teacher choice / LLM-as-judge. Studies use Claude and GPT interchangeably as "strong teachers" for robustness, and caution that GPT-4 favors its own generations as an evaluator [5]. We remove this bias by judging with execution.

On-policy distillation & RLVR. On-policy distillation [1] and contrastive distillation [4] improve imitation; RL-with-verifiable-rewards underlies recent reasoning models. We connect these: the teachers' collaboratively-verified problems become the RLVR reward environment.

Ensemble learning with virtual/synthetic data (historical lineage). The intuition that an ensemble of learners trained with *virtual* — synthetically generated — data can generalize beyond any single member predates the deep-learning era: Jang [2] proposed an ensemble learning algorithm driven by virtual data at POSTECH in 1999. Our setting is a modern realization of the same intuition: the ensemble members are frontier LLM *teachers*, and the "virtual data" is a curriculum of execution-verified synthetic problems. The connection is one of lineage rather than method — the 1999 virtual data augmented ensemble diversity for a single predictor, whereas our verified problems instead define a *reward landscape* for policy optimization (RLVR). We therefore position this work as a spiritual successor to that line: the twist is that we do not average ensemble outputs but use the collaboratively-verified curriculum as a reinforcement-learning environment.

Gap. We find no work that ranks *named frontier models as teachers* by execution, contrasts *imitation vs verifiable-reward collaboration*, and packages it as a *compete-then-collaborate* pipeline.

3 Method

3.1 Execution-verified generation (the judge)

Each teacher receives a task (function signature or a competition problem) and returns reasoning + one code block. We extract the code and run it in an isolated subprocess (rlimits + timeout) against **hidden tests** (unit-test asserts for function tasks; stdin/stdout for competition tasks). Only test-passing solutions are kept. Correctness therefore comes from execution, not from mimicking a teacher.

3.2 Competition with fairness controls

(a) **Shared task bank** — all teachers solve identical problems (function-signature disambiguated; externally-dependent tasks excluded to keep function-only comparison valid). (b) **Self-correction** — failed tasks are returned to the teacher with the failing test error for up to two retries, modeling a teacher who revises (raising each teacher's coverage and removing a "one-shot luck" confound). (c) **Intersection control** — the student-training set is restricted to problems *all* teachers solved, giving identical problems and equal counts, so only solution/explanation *style* differs.

3.3 Collaboration: two modes for one student

The **union** of all teachers' verified solutions forms the collaborative curriculum. We use it two ways: **(SFT)** imitate the pooled solutions; **(RLVR)** treat the verified problems as a GRPO reward environment where reward = fraction of tests passed. Student = Qwen2.5-Coder (7B primary, 32B secondary), LoRA.

4 Experimental Setup

- **Student**: Qwen2.5-Coder-7B / -32B, LoRA (bf16), GB10 128 GB unified.
- **Teachers**: Claude, Codex (GPT), Grok, and Gemini via headless CLIs (four major labs: Anthropic, OpenAI, xAI, Google). Gemini participates in the execution-verified *competition* ranking; the *collaboration* experiments (SFT/RLVR) use the union of the first three teachers' verified solutions.
- **Task banks**: function tasks from MBPP (teaching split; MBPP-*test* held out), bug-fix tasks via mutation of MBPP references; competition problems from `deepmind/code_contests` (difficulty 6–9).
- **Held-out eval (no leakage verified)**: MBPP-test (150) and a disjoint competition set (68). Metric: execution pass@1.
- **RLVR**: TRL GRPOTrainer + PEFT (HF path; Unsloth kernels avoided due to a LoRA-dtype bug); reward = test-pass fraction + small format bonus; HF-generate rollouts (vLLM incompatible on this stack).

5 Results

5.1 Teacher competition (generation pass@1)

Easy (MBPP, 200 shared problems) — saturated.

Hard (`code_contests`, difficulty 6–9, 150 problems) — the informative signal.

Teacher	one-shot	after self-correction
Claude	96.5%	**100%**
Grok	89.5%	99.5%
Codex	90.0%	99.0%
Gemini	85.0%	99.5%

Table 1: Easy MBPP is saturated; all four teachers reach 99–100% after self-correction, likely reflecting benchmark exposure rather than skill.

Teacher	pass@1 (solve)	code-extraction success
Gemini	**77%** (115/150)‡	100%‡
Claude	69% (104/150)§	96% (144/150)
Codex	69% (103/150)	90%
Grok	50% (75/150)†	73% (109/150)†

Table 2: Hard competition problems separate the teachers: Gemini > Claude ≈ Codex > Grok. Gemini leads by ≈8 points; Claude and Codex are within one problem of each other.

Interpretation (v0.2, addressing peer review). MBPP near-ceiling scores (99–100%) most likely reflect *benchmark saturation / training exposure* — MBPP is a public 2021 benchmark seen by all four teachers — **not** differential skill. We therefore do **not** rest the teacher ranking on MBPP.

On the harder `code_contests` set the teachers separate more clearly: **Gemini leads (77%, 115/150), then Claude (69%) ≈ Codex (69%), then Grok (50%)**. No single vendor's model is uniquely best on hard problems; Gemini is ahead by ≈8 points, while Claude and Codex are within one problem of each other. Crucially, the ranking is **not** produced by any LLM judge (execution only), so *self-preference / family bias is excluded by design*; the residual risks are asymmetric train-leak and CLI/parser artifacts (Section 7).

Fairness corrections. We found and corrected three *infrastructure* artifacts (not capability differences), applied identically to each affected teacher and reported transparently:

- **§ Claude** — the first run was interrupted at 122/150 problems. We completed the remaining 28 (22 passed) so every teacher is scored on the same 150, giving Claude a fair **69% (104/150)**, up from an incomplete-run 67% (82/122).
- **‡ Gemini** — the API's prepaid credits were depleted partway through generation, so the final 34/150 problems returned billing errors (429) counted as no-code. After the credits were restored we re-ran exactly those 34 and merged them: **77% (115/150)**, 100% code-extraction.
- **† Grok** — initially 52% of its batch CLI calls returned empty (timeout/flakiness, not wrong code). We added empty-response retries and re-measured: **50% (75/150)**, extraction 48%→73% (109/150). A residual 27% (41/150) still returned no usable code even after retries; we cannot fully separate CLI flakiness from genuine difficulty, so we report it transparently rather than discard those items. The remaining Grok–Codex/Claude gap (50% vs 69%) is thus a conservative (Grok-favorable) estimate.

5.2 Collaboration mode A — imitation (SFT) does not help competent students

Held-out pass@1 (intersection-controlled, 197 shared problems):

Student	base	+Claude	+Codex	+Grok
7B (MBPP)	76.7%	74.0%	70.0%	69.3%
32B (MBPP)	82.0%	80.0%	77.3%	79.3%

Table 3: All SFT students fall *below* base; teacher ranking preserved (Claude least harmful).

All students fall **below base**, and the teacher ranking is preserved (Claude least harmful). The **union** of all teachers (SFT) is also below base (MBPP 72.7%, competition **2.9%** vs base 5.9%).

Imitation at this scale degrades already-competent coder models; the bottleneck is task *difficulty/headroom*, not model size — 7B and 32B both degrade.

5.3 Collaboration mode B — verifiable-reward RL (RLVR) improves the student

Same curriculum, competition held-out (base is weak here → large headroom):

Method	competition base→student	direction
SFT (union)	5.9% → 2.9%	↓ degrade
RLVR (GRPO, 200 steps)	5.9% → 7.4%	↑ +25% relative
RLVR (GRPO, v2, 1000 steps)	5.9% → **8.8%** peak (7.4% @1000)	↑ **+49% rel. at peak**

Table 4: Same data, opposite direction: imitation degrades, verifiable-reward RL improves. The v2 1000-step run peaks at 8.8% (step 250–750) with a mild late regression to 7.4%.

Training reward rose from ≈0 early to 0.25–1.0 late (generalization on train), and held-out improved. **Same data, opposite direction from SFT.**

6 Discussion

The results give a crisp message: **the value of AI-teacher collaboration is not pooling answers to imitate, but jointly building a verifiable environment in which the student learns by doing.** Imitation transfers *style*, which a competent coder already has; RLVR transfers *capability* by rewarding verified success. This also explains the reproduced multi-teacher-KD degradation (knowledge conflict): merging answers cannot exceed the student's imitation ceiling, whereas RL against verifiable rewards can.

7 Limitations

- **Benchmark saturation / train-leak.** MBPP is a widely known 2021 benchmark; near-ceiling teacher scores (99–100%) likely reflect training-set exposure rather than differential capability, and our stable-ranking discussion therefore rests on the harder `code_contests` subset. We do not verify whether individual test items appear verbatim in any teacher's pretraining corpus; asymmetric leakage across teachers would bias the ranking.

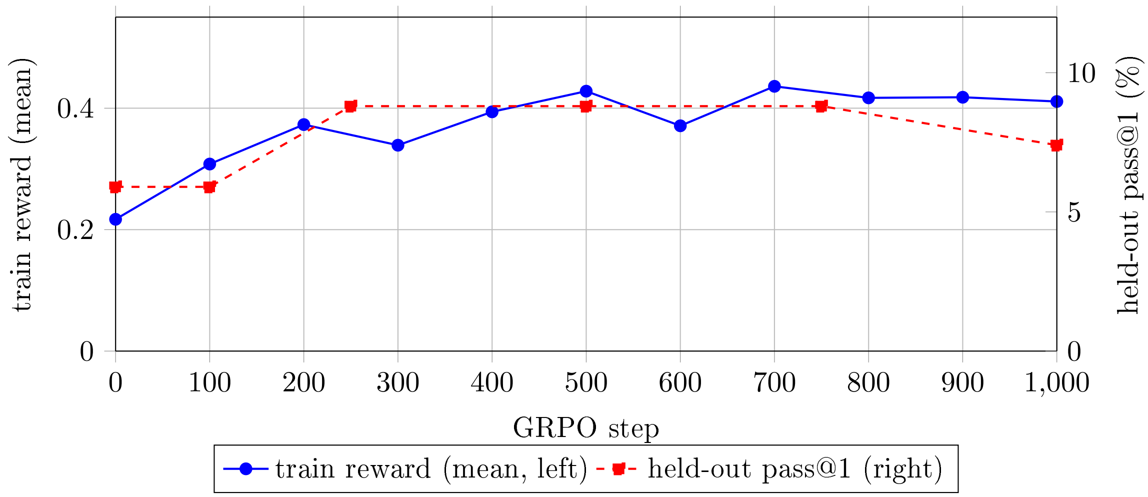

Figure 1: v2 learning curve (1000 steps). Left/blue: logged training reward (mean), rising 0.22 → ≈0.43 then plateauing. Right/red: held-out pass@1 on the 68 competition problems per checkpoint — base 5.9% (4/68), rising to an 8.8% (6/68) plateau over steps 250–750, with a mild regression to 7.4% (5/68) at step 1000. Differences within the plateau correspond to ±1 problem out of 68 (within sampling noise); the robust signal is base → RLVR (4 → 5–6/68).

- **Prompt/parser/CLI symmetry.** All teachers share one prompt template and one code-extraction parser. We found and corrected a fairness artifact where Grok's batch CLI returned empty for 52% of hard-problem calls (timeout/flakiness, not wrong code); after adding empty-response retries the comparison is fair. Residual formatting advantages, if any, are bounded by the shared parser and reported extraction-success rates.

- **The teacher ranking is weaker than the student-side result.** On the hard subset Gemini leads by ≈8 points, while Claude ≈ Codex (within one problem) and Grok trails; the fine ordering below Gemini is only weakly separated given the sample size. By contrast, the **SFT-fails / RLVR-succeeds reversal is robust**: it is a change in the *student's* held-out performance and does not depend on the teacher ranking or on any LLM judge.

- RLVR gains are modest in absolute terms with sparse competition rewards. The 1000-step v2 run improves held-out pass@1 from a 5.9% base to an 8.8% plateau (steps 250–750) with a mild late regression to 7.4% at step 1000; because the held-out set has only 68 problems, differences within the plateau (±1 problem) are within sampling noise, so we report the robust direction (base → RLVR, 4 → 5–6/68) rather than a precise peak, and recommend checkpoint selection over training to the final step. A larger held-out set would tighten these estimates.

- Single student family (Qwen2.5-Coder); one language (Python).

- **Ethics / terms of service.** This is **academic research** — a benchmarking and methodology study — not the development, deployment, or distillation of a commercial model that competes with any provider; no trained model is offered as a product. This purpose places the work outside the *competition-scoped* restrictions of Anthropic (no competing product / competing-model training) and OpenAI (no models that compete with OpenAI); Google's Gemini terms are likewise competition-scoped. We additionally (i) do **not** redistribute any teacher's raw outputs, (ii) do **not** release models trained on teacher outputs, and (iii) base the headline RLVR result on public competition problems with an execution reward — using **no teacher-output distillation** — which also addresses xAI's act-scoped restriction on distilling outputs. Correctness is judged by deterministic execution, not teacher imitation; released tasks/tests/harness let others reproduce every number either offline (re-verify released artifacts) or by regenerating teacher solutions under their own provider agreements.

- Hardware: a GPU GSP timeout (Xid 119/154) required a reboot during a long RLVR run; we mitigate via reduced rollout intensity, checkpointing, and Xid monitoring.

8 Conclusion

Frontier AIs can be ranked as teachers by an unbiased, execution-based judge, and — more importantly — their collaboration is best expressed as a *verifiable curriculum for reinforcement learning*, which improves a coding student where imitation fails. We release the full on-prem pipeline and the framework patches required to reproduce it on NVIDIA GB10.

Reproducibility

All code, task banks, hidden tests, results, and this paper are released at `https://github.com/shawnkim678/compete-then-collaborate`. Readers can re-verify every reported number offline (`python reproduce.py -check-banks -selftest` on the released artifacts) or regenerate teacher solutions with their own API keys under their own provider terms; we redistribute no teacher outputs (see the ethics statement).

Scripts: `verify_code.py`, `verify_stdio.py`, `build_taskbank_mbpp.py`, `build_contests_bank.py`, `prof_run.py`, `prof_run_stdio.py`, `prof_retry.py`, `equalize_golden.py`, `eval_code_students.py`, `grpo_train.py`. Orchestration (setsid): `grpo_orchestrator.sh`. GB10 framework patches (trl import flags, vllm-off, `PreTrainedModel.warnings_issued`, HF+PEFT to avoid the Unsloth LoRA-dtype bug) documented in the research log §16–17.

Acknowledgments

We thank Dr. Min Jang (Pohang University of Science and Technology, POSTECH), whose 1999 dissertation on ensemble learning with virtual data [2] provided the conceptual lineage for the verifiable-curriculum framing developed here. In his review of an earlier draft, Dr. Jang highlighted the engineering contributions of the reported system as its principal strengths — the stack-level debugging on the NVIDIA GB10 platform, the CLI orchestration across four commercial LLMs, and the execution sandbox. No third-party copyrighted material is redistributed with this paper; the referenced dissertation is cited bibliographically only.

References

[1] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Min Jang. *Ensemble Learning Algorithm using Virtual Data.* Ph.D. dissertation, Pohang University of Science and Technology (POSTECH), Pohang, Republic of Korea, November 1999. Advisor: Cheeha Kim.

[3] Ruihan Jin, Pengpeng Shao, Zhengqi Wen, Jinyang Wu, Mingkuan Feng, Shuo Yang, Chu Yuan Zhang, and Jianhua Tao. Exploring knowledge purification in multi-teacher knowledge distillation for LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. Reports that distillation performance can decline as the number of teacher LLMs grows (knowledge conflict), motivating rationale purification.

[4] Jongwoo Ko, Tianyi Chen, Sungnyun Kim, Tianyu Ding, Luming Liang, Ilya Zharkov, and Se-Young Yun. DistiLLM-2: A contrastive approach boosts the distillation of LLMs. *arXiv preprint arXiv:2503.07067*, 2025.

[5] Nathan Lambert. *Reinforcement Learning from Human Feedback: Alignment and Post-Training of LLMs*. Manning Publications, 2025. ISBN 9781633434301. Also available as arXiv:2504.12501. Discusses LLM-as-judge self-preference bias (ch. 12).